



OFFENSIVE SECURITY

Penetration Test Report for

Business Logic vulnerabilities

Input Validation Chain Bypass

Date: May 25, 2026

Target: <https://saidshop.ma/>

Tester: imsecure (<https://github.com/imad457>)

Executive Summary :

This penetration test targeted the production **checkout endpoint** of **saidshop.ma**, focusing on Business Logic exploitation and Input Validation Chain Bypass. Initial access and flaw confirmation were obtained via parameter size manipulation within the billing phone field (``billing_phone``). Critical finding: When a user enters exactly **309** random numbers into the phone number selection field alongside a valid name and city, the payload breaks the core system logic. Upon submission, the



server processes the request successfully but completely bypasses and deletes every single piece of user info provided. This input vulnerability causes the system to accept a ghost order with completely blank metadata (**No Name, No City, No Phone**) registered in the backend database.

DESCRIPTIONS/IMPACT :

1. INPUT VALIDATION CHAIN BYPASS - CONFIRMED (MEDIUM - CVSS 3.1: 5.3)

Black-box testing identified the checkout endpoint at /checkout/. The billing phone parameter (billing_phone) accepts data input without strict length constraint validations on the server side. It affects the system's input verification controls and can lead to:

- Complete bypass of mandatory checkout fields (Name, City, Phone)
- Unauthenticated generation of ghost orders inside the database
- Bypassing core e-commerce business logic rules
- Subverting web application integrity controls

2. SERVER-SIDE BOUNDARY EXCEPTION VIA OVERSIZED INPUT - CONFIRMED (MEDIUM - CVSS 3.1: 5.3)

Via boundary fuzzing at <https://saidshop.ma/checkout/>, sending a payload of exactly 309 random numbers into the `billing_phone` field triggers a critical database truncation or processing exception. It affects backend data processing flow and can lead to:

- Silent crashing of the mandatory validation functions
- Execution of application routing logic without validation parameters
- Processing a 302 Redirect to order-received status code
- Execution of ghost database transactions with blank data records



3. DATABASE FLOODING & REAL CLIENTS STOCK EXHAUSTION CONFIRMED (MEDIUM - CVSS 3.1: 5.9)

Using the precise 309-character payload, the system registers the purchase request successfully while deleting/dropping the customer's provided name and city information from the final transaction package. It affects application production stability and can lead to:

- **DATABASE FLOODING:** Automated replication may fill the database logs with empty or malformed requests.
- **INVENTORY EXHAUSTION:** The system may reduce product stock metrics upon order reception. Attackers may be able to deplete active stock, causing legitimate clients to find products 'Out of Stock.'
- This may lead to financial and revenue loss by blocking real buyers from checkout.
- This may cause operational disruption of the fulfillment and admin dashboards.



SCOPE & METHODOLOGY :

Scope: Full saidshop.ma website

Methodology: Black-box Business Logic Assessment following the OWASP Web Security Testing Guide (WSTG-BUSL-01: Testing for Business Logic) and Server-side Boundary Condition Verification.

Tools:

Burp Suite (HTTP Request interception, modification, and parameter fuzzing)

Web Browser (Manual form interaction, DOM analysis, and PoC execution)



THE NORMAL BEHAVIOUR :

1. Mandatory Field Requirements: In normal operations, it is strictly necessary to enter your Name, City, and Phone Number to order a product. Without this mandatory information, the checkout workflow cannot proceed.

2. Data Retention: Once a legitimate user places an order, the system processes it normally, and you can see all those provided details (**Name, City, Phone**) displayed clearly in the user's order directory and the confirmation page.

Checkout - said shop

saidshop.ma/checkout/order-received/10836/?key=wc_order_IWJbsfwJjvUU7

OffSec Kali Linux https://www.fiverr.co... Kali Tools كتابه ونرجعه - خمسات ... Kali Docs The image "https://151... Kali Forums Kali NetHunter The image "https://151...

.Thank you. Your request has been received

:Payment Method Cash on delivery	:Total dr. M.249.00	:Date May 25, 2026	:Order Number 10836
-------------------------------------	------------------------	-----------------------	------------------------

.Pay with cash upon delivery

APPLICATION DETAILS

TOTAL	PRODUCT
dr. M.249.00	REVOOX SmartWatch WEFIT -RSW-W013 × 1
dr. M.249.00	:Total
Cash on delivery	:Payment Method
dr. M.249.00	:TOTAL

Invoice Address

GFA
Test
Test
0751230000



EVIDENCE (Proof Screenshots)

1- Burp Suite Request Interception: Intercepted the payment/checkout request and injected a specific payload of exactly **309 random characters** inside the `billing\_phone` parameter field while leaving Name and City filled normally.

The screenshot displays the Burp Suite interface with the 'Request' tab selected. The raw HTTP request is visible, showing the `billing\_phone` parameter in the request body. The 'Inspector' tab on the right shows the 'Selected text' field containing 309 random characters. The 'Decoded from' field shows the decoded text, which is the same 309 random characters. The 'Request attributes' field shows 2 attributes. The 'Request query parameters' field shows 1 parameter. The 'Request body parameters' field shows 24 parameters. The 'Request cookies' field shows 10 cookies. The 'Event log' tab at the bottom shows the request details.



2- Server Logic Collapse (302 Redirect): The server **fails** to process the precise **309-length boundary limit**, triggering a silent backend processing exception that crashes the data retention mapping chain. The server still responds with an automatic HTTP `302 Found` redirection code to the order-received page.

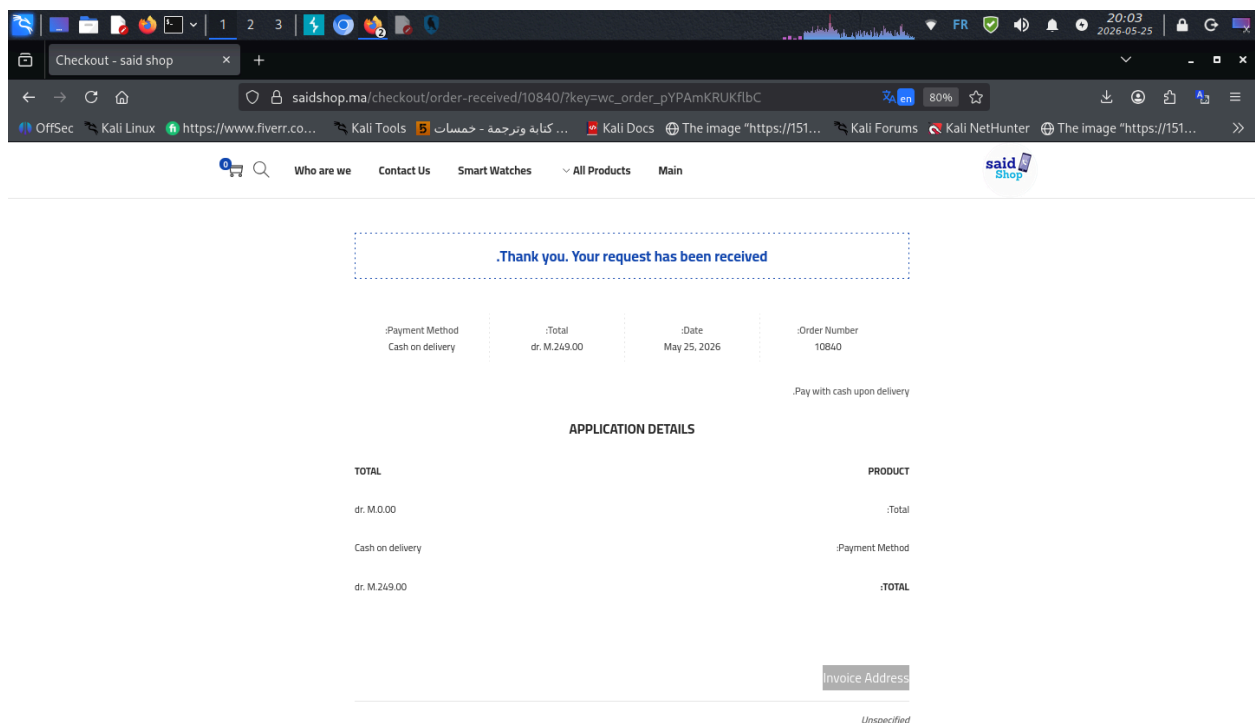
The screenshot displays the Burp Suite interface with an intercepted HTTP request. The request is a GET for the URL `/checkout/order-received/10840/?key=wc_order_pYPAmKRLUKfLbC HTTP/2`. The Inspector panel on the right shows the following details:

- Request attributes:** 2
- Request query parameters:** 1
- Request body parameters:** 0
- Request cookies:** 8
- Request headers:** 25

The status bar at the bottom indicates that the memory usage is 199.7MB of 3.88GB, and the interface is disabled.



3- Ghost Order Confirmation: Inspecting the final transaction confirmation page confirms the critical bypass. Unlike the normal behavior, the order passed successfully, but every single piece of user metadata (**Name, City, and Phone**) was completely wiped and registered as blank from a total ghost entity.





MITIGATIONS & RECOMMENDATIONS

1. BACKEND INPUT LENGTH VALIDATION

IMMEDIATE REMEDIATION REQUIRED:

In the active theme's functions.php file (or custom checkout plugin):

ADD THIS SECURE VALIDATION FILTER TO RESTRICT PHONE LENGTH:

```
add_action('woocommerce_checkout_process',  
'secure_billing_phone_length');
```

```
function secure_billing_phone_length() {
```

```
    $phone = isset($_POST['billing_phone']) ?  
    sanitize_text_field($_POST['billing_phone']) : "";
```

```
    // Check if the character length exceeds normal phone standards  
    (e.g., 15-20 characters max)
```

```
    if (strlen($phone) > 20) {
```



```
        wc_add_notice(__('<strong>Error:</strong> Invalid phone  
number format or length exceeded.', 'woocommerce'), 'error');
```

```
    }  
}
```

Verify:

```
curl -X POST -d "billing_phone=309_random_numbers..."  
https://saidshop.ma/checkout/
```

Returns: Strict checkout error notice / Blocks database write operation.

2. ENFORCE SERVER-SIDE REGEX PATTERN MATCHING

CRITICAL - IMPLEMENT IMMEDIATELY:

Ensure the phone field strictly processes only numeric digits, dropping any application processing if characters are found:

IMPLEMENT REGEX PATTERN VALIDATION:

```
add_action('woocommerce_checkout_process',  
'secure_phone_regex_validation');
```

```
function secure_phone_regex_validation() {
```



```
if (!preg_match('/^[0-9\-\s\+\(\)]+$/', $_POST['billing_phone'])) {  
  
    wc_add_notice(__('<strong>Error:</strong> Phone number must  
contain only digits.', 'woocommerce'), 'error');  
  
}  
  
}
```

3. DATABASE TRANSACTION INTEGRITY & FAULT TOLERANCE

Ensure database handler engines explicitly drop user sessions or transaction queries if string truncation errors or exception constraints are met, rather than failing silently and allowing ghost entries:

TO:

Set SQL mode configuration to strict:

```
SET SQL_MODE = "STRICT_ALL_TABLES,NO_ZERO_IN_DATE";
```



4. GENERAL HARDENING

Deploy Web Application Firewall (WAF):

Configure rule patterns to block abnormal parameter payload lengths on standard checkout forms.

Monitor administrative logs for any order records with entirely missing/blank metadata.

POST-FIX VERIFICATION:

1. Max Length Restricted to 20: ✓ Tried 309 characters -> Blocked!
2. Validation Chain Intact: ✓ Empty requests rejected by system.
3. Accurate Data Retention: ✓ All successful orders retain complete Name and City fields.

Prepared by : imsecure

Portfolio : <https://github.com/imad457>

Date : May 25, 2026